AI-Assisted Host-Based Forensic Report Generation & Artifact Correlation

Group 10: Aadi Jain,
Supavit (Troy) Pitaknarongphorn, Evan Yu

Introduction

The Problem:

- Digital forensics requires tremendous amount of manual effort and time, causing the process to be slow, error-prone, and impossible to finish within the timeline, in some cases. Analysts must sift through massive tool outputs with limited automation

Our Proposal:

- Build an AI-assisted pipeline to help speed up the process
- Generates cohesive, investigator-ready reports
- Reduce laborious tasks demanded from analysts

Literature Reviews

Enhancing Digital Investigation: The Role of Generative AI (ChatGPT) in Evidence Identification and Analysis in Digital Forensics

- **Goal:** Use ChatGPT (GPT-4V) to analyze chats and images, extract digital evidence, and generate forensic reports.
- **Methodology:** Collect and clean data → input into AI → identify evidence (dates, actors, intent) → perform forensic analysis → generate structured reports.
- **Key Findings:** ~84–86% accuracy, ~42% faster analysis, effective at detecting crimes and extracting timelines and behaviors.

Digital Forensics in the Age of Large Language Models

- **Goal:** Examine how Large Language Models (LLMs) are transforming digital forensics, including evidence analysis, automation, and investigator workflows.
- **Methodology:** Survey and analyze emerging uses of LLMs in forensics (e.g., evidence interpretation, report generation, automation pipelines).
- **Key Findings:** LLMs can significantly enhance efficiency and automation in forensic investigations

Challenges

Enhancing Digital Investigation: The Role of Generative AI (ChatGPT) in Evidence Identification and Analysis in Digital Forensics

- AI can produce incorrect, incomplete, or biased interpretations of evidence.
- General-purpose LLMs lack deep forensic expertise unless fine-tuned.
- Sensitive forensic data (messages, images) may be exposed when using AI systems.

Digital Forensics in the Age of Large Language Models

- ChatGPT does not clearly explain how it reaches conclusions, making it difficult to trust or validate results.
- AI-generated evidence must meet strict legal standards (e.g., chain of custody, auditability).
- Risk of reduced critical thinking in analysis, causing other kinds of human-error

Originality & Relevance

- Introduce an MCP-based architecture
  - Help eliminate hallucinations and bias
- Enable cross-artifact correlation
  - Provide more context and accuracy
- Introduce fine-tuning mechanism
  - Give the model relevant skills to perform the task.

Research Question:

Can generative AI **reliably** produce accurate, structured forensic reports from tools like Autopsy—and improve investigations through cross-artifact correlation?

MCP - Model Context Protocol

- Protocol for connecting LLMs to tools
- A server hosts a set of endpoints that the AI can invoke to retrieve information that it can base its response on
- Alternatives: spoonfeed or have the LLM find for itself
- The more steps the LLM has to perform, the less data the LLM has been provided, the more likely the final result will be wrong
- Let’s provide concretize some of the steps for the LLM so that it doesn’t hallucinate





Autopsy MCP

- Comprehensive forensics tool
- Stores results in a sqlite DB (easy to interface)

```python
@mcp.tool()
def get_events_around_time(timestamp: int, window_seconds: int = 3600, limit: int = 100) -> List[Dict[str, Any]]:
    """Get events from the timeline within a window around a specific timestamp."""
    try:
        start_time = timestamp - window_seconds
        end_time = timestamp + window_seconds
        with get_db_connection() as conn:
            cursor = conn.cursor()
            query = """
                SELECT ev.event_id, ev.time, type.display_name as type, descr.full_description
                FROM tsk_events ev
                JOIN tsk_event_types type ON ev.event_type_id = type.event_type_id
                JOIN tsk_event_descriptions descr ON ev.event_description_id = descr.event_description_id
                WHERE ev.time BETWEEN ? AND ?
                ORDER BY ev.time ASC
                LIMIT ?
            """
            cursor.execute(query, (start_time, end_time, limit))
            return [dict(row) for row in cursor.fetchall()]
    except sqlite3.OperationalError as e:
        return [{"error": f"Timeline tables not found or schema differs: {e}"}]
```

MCP Endpoints

def list_artifact_types() -> List[Dict[str, Any]]:

def list_artifacts(artifact_type_name: str, limit: int = 100) -> List[Dict[str, Any]]:

def get_artifact_details(artifact_id: Union[int, str]) -> List[Dict[str, Any]]:

def run_keyword_search(regex: str, limit: int = 100) -> List[Dict[str, Any]]:

def get_events_around_time(timestamp: int, window_seconds: int = 3600, limit: int = 100) -> List[Dict[str, Any]]:

def search_files_by_name(name_pattern: str, limit: int = 100) -> List[Dict[str, Any]]:

def get_file_info(obj_id: int) -> Dict[str, Any]:

def get_registry_key(hive_name: str, key_path: str) -> List[Dict[str, Any]]:

def get_web_history(limit: int = 100) -> List[Dict[str, Any]]:

def get_installed_programs(limit: int = 100) -> List[Dict[str, Any]]:

def **query_db**(sql: str, params: List[Any] = None) -> List[Dict[str, Any]]:

Architecture / Workflow

1. Run autopsy
2. Configure the MCP
   a. ~/.gemini/settings.json
3. Run Claude Desktop or Gemini CLI and explicitly prompt it to use this MCP

```
"mcpServers": {
  "autopsy": {
    "command": "/home/evan/CMU/autopsy-mcp/.venv/bin/python",
    "args": [
      "/home/evan/CMU/autopsy-mcp/server.py"
    ],
    "env": {
      "AUTOPSY_DB_PATH": "/home/evan/CMU/autopsy-mcp/autopsy.db"
    }
  }
}
```

